

ClinicOps — Interview preparation

Use this doc to **describe what you built**, **sound credible about ownership**, and **explain architecture** without sounding like you only followed a tutorial.

1. What this project is (plain language)

ClinicOps is a **clinic operations** web product: multiple clinics can use the system, each with its own staff and patients. A **SuperAdmin** approves new clinics. After approval, a **ClinicAdmin** manages users (**Doctors**, **Nurses**, **Lab technicians**). At reception, staff **register patients** and open a **patient case** (a visit). **Nurses** record **vital signs**; **Doctors** write the **medical report** (anamnesis, diagnosis, therapy). The UI can **update in real time** when vitals or reports change (**SignalR**). The backend can **generate a PDF case report** (with doctor **signature** and **stamp** images) via **PuppeteerSharp**.

In one sentence for an interviewer:
“It’s a multi-tenant clinic operations app: JWT auth and roles, EF Core + MySQL, a React dashboard that talks to a .NET 8 API, real-time case updates over SignalR, and server-side PDF reports.”

2. How the repo is split

Part	Path	Role
Backend	clinicops-server/clinicops/	ASP.NET Core Web API, Identity + JWT, EF Core, SignalR hub, PDF service
Frontend	clinicops-client/	React (CRA), React Router, Axios, Tailwind, SignalR client

There is already a deep backend write-up in the repo: clinicops-server/clinicops/docs/PROJECT-OVERVIEW-FOR-INTERVIEW.md and API notes in docs/API-FRONTEND.md. This preparing.md focuses on **how you present** the work in an interview.

3. Architecture you should be able to draw or narrate

Request path (typical):
Browser → **React** (src/api/*.js, shared **Axios** instance with Authorization: Bearer ...) → **Controller** → **Service** and/or **DbContext** → **MySQL**.

Auth:
ASP.NET Core **Identity** stores users and roles. **JWT** is issued at login; claims include **roles** and, for clinic users, **clinicId** / **clinicName** so queries stay **tenant-scoped**. SignalR uses the same token (often via **?access_token=** on the hub URL — see Program.cs).

Real-time:
ClinicHub + **IHubContext<ClinicHub>** from controllers: after vitals/report/status changes, the server **pushes** events (e.g. vitals updated, report updated) to **groups** keyed by clinic or case. The React app uses **@microsoft/signalr** (see src/context/SignalRContext.jsx, src/services/signalr.js).

PDF:
CaseReportPdfService builds **HTML** from case data (patient, vitals, report, doctor display name, signature/stamp URLs), then **PuppeteerSharp** renders it to PDF bytes.

Frontend structure (high level):
App.js defines routes; **ProtectedRoute** gates /dashboard/*. **AuthContext** holds user/token; **DashboardLayout** + **Sidebar** organize role/panel flows. Feature areas live under src/pages/Dashboard/ (cases, patients, applies, profiles, laboratory, services, etc.) with small **API modules** per domain in src/api/.

You do **not** need to quote every folder name — you need the **story**: *layered API, scoped data, JWT, hub for live updates, HTML→PDF for reports*.

4. Ownership — how to talk about it honestly and strongly

Interviewers want to know **what you decided, built, or debugged**, not that a repo exists.

Strong phrases (use only what is true for you):

- “I owned the [patient case flow / SignalR integration / PDF pipeline / clinic apply–approve flow / doctor profile uploads / …].”
- “I chose [JWT + claims for clinicId] because the API is stateless and we needed per-clinic isolation without server sessions.”
- “When X broke, I traced it through [controller → service → EF / React → API / hub negotiate], and the fix was …”

If you worked across stack:
“I implemented the nurse/doctor case screen end to end: API endpoints, DTOs, and the React detail view, including subscribing to SignalR so the doctor sees vitals without refresh.”

If you mostly did one side:
Say that clearly — “I focused on the backend; the frontend was existing / pair work” — then **go deep** on your side (migrations, authorization rules, hub groups, PDF edge cases).

Concrete prep: Pick **2–3 flows** you can walk through step by step (login → token claims → one authenticated call; register patient → case created; save vitals → SignalR event; open PDF endpoint). Open the code once before the interview so the walk is **specific**, not generic.

5. Real experience vs “tutorial clone” — how interviewers tell the difference

Signals of a real project (you should be able to demonstrate):

- **Tradeoffs:** Why JWT? Why SignalR instead of polling? Why Puppeteer for PDF vs something else?
- **Failure modes:** Invalid/expired token, wrong role, missing `clinicId`, CORS, hub reconnect, file upload size/type, DB migration drift.
- **Your edits:** A bug you fixed, a feature you added, a refactor you did — with **before/after** reasoning.

If you learned from a course or template:

That is fine — say *“I started from [course baseline] and then added/changed ...”* and list **your** additions (multi-tenancy rules, lab/services screens, report PDF, etc. — whatever actually applies in **your** git history or memory).

Avoid: Memorizing jargon without being able to point to **one** controller, **one** React page, and **one** database table involved in a story.

6. Ability to explain architecture (compact scripts)

~30 seconds

“ClinicOps is a multi-tenant clinic app. The backend is .NET 8: EF Core on MySQL, Identity for users, JWT for API auth with clinicId in claims for isolation. Controllers call services and DbContext; SignalR notifies the React client when vitals or reports change. PDFs are generated on the server from HTML via Puppeteer.”

~60 seconds (add detail)

Add: *“SuperAdmin approves clinic applications; clinic staff manage users. Reception creates patients and cases; nurses vitals, doctors reports. The React app uses Axios with the bearer token and a SignalR connection to the clinic hub. Static uploads (logos, signatures) live under wwwroot and are referenced in reports.”*

7. Likely questions — short answer bank

Topic	What to say
Multi-tenancy	Data is scoped by ClinicId ; token carries clinicId for normal users; queries filter by tenant.
Why SignalR?	Push updates to doctor/nurse UIs when shared case data changes; avoids polling and keeps views in sync.
Why JWT?	Stateless API auth; works for SPA + SignalR ; scales without server session storage.
PDF approach	HTML template + headless Chrome (PuppeteerSharp) for reliable layout and images (signature/stamp).
Roles	SuperAdmin, ClinicAdmin, Doctor, Nurse, LabTechnician (etc.) — Authorize attributes on controllers.
Frontend stack	React, React Router, Tailwind, Axios, SignalR client — SPA on localhost:3000 , CORS configured on API.

8. Before the interview — quick checklist

- ☐ Run API + client locally; log in as **two roles** if you can (e.g. nurse + doctor) and trigger **one SignalR** update.
 - ☐ Open **one** controller and **one** service and explain the flow in your own words.
 - ☐ Skim PROJECT-OVERVIEW-FOR-INTERVIEW.md for entity names (**PatientCase**, **VitalSigns**, **MedicalReport**).
 - ☐ Prepare **one** story: a problem you solved (performance, auth, UI bug, migration, hub reconnect).
-

9. Integrity reminder

Only claim **ownership** of work you actually did. Everything else, frame as *familiar with*, *reviewed*, or *would approach by* Interviewers respect accuracy more than overstated titles.

Good luck — ClinicOps is a **credible full-stack portfolio story** if you can narrate flows, decisions, and a few concrete code paths.